

DevCollab

Full Project Specification & Strict 30-Day Execution Plan

Status	Week 1 — Authentication module complete. Resuming today.
Plan Starts	Friday, July 3, 2026
Plan Ends	Saturday, August 1, 2026 (Day 30)
Format	30 daily task blocks, grouped into 4 weekly gates
Rule	No day starts until the previous day's Definition of Done is met

This edition contains the complete project reference (Part I) and a strict, dated, day-by-day build schedule (Part II) from July 3 through August 1, 2026. Part III is an execution appendix — weekly gate checklist and hardening checklist. Treat Part II as a contract with yourself, not a suggestion.

PART I

Project Overview

The full reference specification DevCollab is built against.

PART I — OVERVIEW

SECTION 01

Project Vision

DevCollab is a production-inspired software collaboration platform built specifically for developers and small software teams. The purpose is not to build another CRUD app, but to learn how modern full-stack SaaS applications are designed, developed, secured, deployed, and scaled — while staying small enough to build in a month. It simulates concepts found in GitHub, Jira, Linear, Slack, Notion and Discord. Every feature exists because it teaches an engineering concept; the objective is not to finish quickly, it is to become capable of designing and building a complete software product independently.

Personal Goal

By completion: *“I understand how a real production MERN application is built, from database design to deployment.”* Concepts covered:

- System Architecture & API Design
- Authentication & Authorization & RBAC
- MongoDB Modeling
- Redux Architecture & State Management
- File Upload Systems
- Real-Time Communication (WebSockets)
- WebRTC
- Payment Integration
- AWS Deployment, Docker & Redis
- CI/CD
- Production Security & Scaling

PART I — OVERVIEW

SECTION 02

Technology Stack

Layer	Technologies
Frontend	React, TypeScript, Vite, Redux Toolkit (hand-written slices, no generators), React Router, Axios, Tailwind CSS, React Hook Form, Zod, Framer Motion
Backend	Node.js, Express.js, TypeScript, Mongoose, JWT + Refresh Tokens, bcrypt, Express Validator, Socket.IO — every middleware / controller / service / repository handwritten
Database	MongoDB — Users, Projects, ProjectMembers, Tasks, Comments, Messages, Notifications, ActivityLogs, Subscriptions, Payments, Invitations, Attachments, Meetings, Sessions, RefreshTokens
Cloud	AWS S3 (attachments), AWS EC2 (deploy), Docker (containers), Nginx (reverse proxy), Redis (cache, socket scaling, sessions)
Payments	Stripe — freemium, 5 free projects, checkout + webhooks for upgrade

Layer	Technologies
Real-Time	Socket.IO — chat, typing, presence, notifications, activity feed, task/project events
Meetings	WebRTC — video, audio, screen share, meeting chat, mute, camera toggle

React Query may be explored later; the initial version intentionally uses Redux Toolkit for manual state management depth. No boilerplates. No generators. Every Redux slice is hand-written.

[PART I — OVERVIEW](#)
[SECTION 03](#)

Database Collections

No unnecessary duplicated data — references are used wherever relationships naturally exist.

Collection	Collection	Collection
Users	Projects	ProjectMembers
Tasks	Comments	Messages
Notifications	ActivityLogs	Subscriptions
Payments	Invitations	Attachments
Meetings	Sessions	RefreshTokens

[PART I — OVERVIEW](#)
[SECTION 04](#)

Application Modules

Module	Module
Authentication	User Profiles
Projects	Project Members
RBAC	Tasks
Task Comments	Task Attachments
Project Chat	Notifications
Activity Feed	Meetings
Payments	Subscriptions
Admin Dashboard	Future Analytics

User Roles & Permissions

Permissions never depend on role alone — every action is checked through dedicated permission middleware, not a role flag.

Role	Permission Boundary
Owner	Full control — delete project, manage billing, remove admins
Administrator	Broad management — cannot delete owner or transfer ownership
Project Manager	Coordinates tasks & members within a project
Developer	Works on tasks — cannot manage members or delete project
Tester	Can create bug reports — cannot manage project settings
Guest	Read-only access

Core System Flows

Master Project Flow

Register → Verify Email → Login → Dashboard → Create Project → Invite Members → Assign Roles → Create Tasks → Collaborate → Chat → Meet → Upload Files → Notifications → Complete Project

Authentication Flow

Register → Email Verification → Login → JWT → Refresh Token → Redux → Protected Routes → Refresh Middleware → Logout → Invalidate Refresh Token

Project Creation Flow

Create Project → Owner Becomes Project Owner → ProjectMember Created → Activity Log Generated → Dashboard Updated → Notification Generated

Member Invitation Flow

Owner Invites → Invitation Created → Email Sent → User Accepts → ProjectMember Created → Notification Generated → Activity Log Updated

Task Flow

Create Task → Assign Member → Priority → Status → Comments → Attachments → Completion → Activity Feed → Notification

Chat Flow

Open Project → Socket Connect → Join Room → Receive Messages → Typing Indicator → Read Receipts → Store Messages → Live UI Update

Notification Flow

Task Assigned → Notification Created → Socket Event → Notification Popup → Stored In Database → Mark Read

Meeting Flow

Create Meeting → Generate Room → Socket Signaling → WebRTC Connection → Video → Audio → Screen Share → Meeting Ends

Subscription Flow

Create Projects → Count ≤ 5 → Allow → Count > 5 → Stripe Checkout → Stripe Webhook → Subscription Updated → Unlimited Enabled

File Upload Flow

Upload File → Backend Validation → AWS S3 Upload → URL Returned → Database Stores URL → Frontend Displays File

[PART I — OVERVIEW](#)
[SECTION 07](#)

API Architecture & Folder Structure

Every endpoint follows the same pipeline, front to back:

Client → Redux Slice → Axios Service → Express Route → Validation → Auth Middleware → Permission Middleware → Controller → Service → Repository → MongoDB → Response Formatter → Redux Store → React UI

Backend Folder Structure

```
src/  
  config/ · database/ · middlewares/ · routes/  
  controllers/ · services/ · repositories/ · models/  
  validators/ · socket/ · utils/ · constants/  
  types/ · jobs/ · storage/
```

Frontend Folder Structure

```
src/  
  app/ · features/ · components/ · layouts/  
  pages/ · hooks/ · services/ · store/  
  utils/ · types/ · constants/ · assets/
```

[PART I — OVERVIEW](#)
[SECTION 08](#)

Engineering Principles

- Never rush. Never skip validation. Never duplicate business logic.
- Keep controllers small — move business logic into services.
- Repositories only access the database.
- Middlewares perform only middleware responsibilities.
- Every request returns a consistent response shape; every API handles errors explicitly.
- Every feature has loading, empty, and error states.
- Always design with scalability in mind.

[PART I — OVERVIEW](#)
[SECTION 09](#)

Future Versions (Out of Scope for the 30 Days)

Version	Planned Scope
v1.1	Kanban board, drag and drop, sprint management, calendar
v1.2	Markdown wiki, documentation, version history
v1.3	AI features — task descriptions, meeting summaries, insights, chat assistant
v2.0	Team analytics, reports, organization dashboard, audit logs, admin portal

Any idea that surfaces during the 30 days that isn't in the current week's scope goes here — not into the build.

PART II

30-Day Execution Plan

Strict, dated, day-by-day. Starts July 3, 2026. Ends August 1, 2026.

PART II — EXECUTION
SECTION 10

Rules of Engagement

- This plan starts **tomorrow, Friday July 3, 2026**, because Auth (Week 1, Day 0) is already complete.
- Each day has a fixed scope and a **Definition of Done (DoD)**. A day is not finished until its DoD is met — not when the clock runs out.
- No skipping ahead. Day N+1 does not start until Day N's DoD is checked off.
- Miss a day entirely? It gets absorbed into the next day. The 30-day window does **not** extend — cut scope from that day's "nice to have" edges, not from the DoD.
- No new features outside the current day's scope. Ideas go into the Future Versions backlog (Part I, Section 09), not into the code.
- Each week ends in a **Gate** (Days 7, 14, 21, 28/29) — a hard regression + integration checkpoint. If a Gate isn't met, the next week does not start.
- Git-tag the end of every week (week1-complete ... week4-complete). This is your evidence trail.

PART II — EXECUTION
SECTION 11

The Daily Plan

WEEK 1 — FOUNDATION — Jul 03 to Jul 09, 2026

DAY 01

Fri, Jul 03

Auth Hardening & Profile Kickoff

- Audit existing auth: token expiry, refresh rotation, bcrypt cost factor, validation coverage
- Add rate limiting to login/register (express-rate-limit)
- Write a manual test collection (Postman/Thunder Client) covering every auth endpoint
- Design extended User/Profile schema (bio, avatar, socialLinks, skills)
- Build GET /users/me and PATCH /users/me through the full Route→Controller→Service→Repository stack

DEFINITION OF DONE — Auth endpoints pass the full manual test collection; profile update persists with proper validation errors.

DAY 02

Sat, Jul 04

Profile UI & Redux Integration

- Build profileSlice (fetch, update, loading/error states)
- Build ProfilePage UI — editable fields, avatar placeholder, RHF + Zod validation
- Wire Axios profile service through the existing interceptor stack
- Add optimistic update on save with rollback on failure

DEFINITION OF DONE — Profile page loads real data, edits persist after refresh, invalid input shows inline errors.

DAY 03

Sun, Jul 05

Project Schema & Project CRUD (Backend)

- Design Project schema: name, description, visibility, ownerId, createdAt, memberCount
- Design ProjectMember schema with compound unique index {projectId, userId}
- Build full CRUD stack (Create/Read/Update/Delete) for Projects
- Auto-create ProjectMember(owner) + ActivityLog entry on project creation

DEFINITION OF DONE — Can create/read/update/delete a project via API; owner membership and activity log auto-provision correctly.

DAY 04

Mon, Jul 06

Project CRUD (Frontend) & Dashboard Skeleton

- Build projectSlice (list, create, update, delete, currentProject)
- Build ProjectCard component (GitHub repo-card style, per existing design pattern)
- Build Dashboard page: project grid + create-project modal
- Wire loading / empty / error states end-to-end

DEFINITION OF DONE — Dashboard lists real projects from the API; create/delete reflected instantly without a manual refresh.

DAY 05

Tue, Jul 07

RBAC & Permission Middleware

- Define role enum: Owner / Admin / PM / Developer / Tester / Guest
- Build capability-based permission middleware (canDeleteProject, canManageMembers, etc.) — not raw role checks
- Apply middleware to every project-mutating route
- Manually verify the permission matrix for at least 5 actions across all 6 roles

DEFINITION OF DONE — Unauthorized actions return 403 consistently; permission matrix verified role-by-role and documented.

DAY 06

Wed, Jul 08

Member Invitations

- Design Invitation schema: projectId, invitedEmail, role, status, token, expiresAt
- Build invite-member and accept-invitation endpoints
- Build InviteMemberModal + PendingInvitations UI
- Fire ActivityLog entry + Notification stub on invite and on accept

DEFINITION OF DONE — Full invite → accept flow works end-to-end (token link can be simulated manually for now).

DAY 07

Thu, Jul 09

■ WEEK 1 GATE — Integration, Testing & Buffer

- Full regression: auth → profile → project → invite → RBAC, zero console errors
- Fix every bug found this week; refactor any duplicated logic into services
- Write the Week 1 section of the README (architecture decisions made)
- git tag week1-complete

DEFINITION OF DONE — GATE: every Week 1 roadmap item is functionally working end-to-end. No critical bugs open before Week 2 starts.

WEEK 2 — PROJECT MANAGEMENT — Jul 10 to Jul 16, 2026

DAY 08

Fri, Jul 10

Task Schema & Task CRUD (Backend)

- Design Task schema: title, description, projectId, assigneeId, priority, status, dueDate, createdBy
- Build full CRUD stack with permission checks (members create; assignee/PM update status)
- Add compound index on {projectId, status} for query performance

DEFINITION OF DONE — Task CRUD works via API; indexed queries confirmed (via .explain() or query logs).

DAY 09

Sat, Jul 11

Task UI & Redux

- Build taskSlice (CRUD + active filters)
- Build TaskList, TaskCard, TaskDetail components
- Priority/status badges, due-date picker

DEFINITION OF DONE — Tasks visible per project; create/update/delete reflected live in the UI.

DAY 10

Sun, Jul 12

Comments & Attachments (Schema + Backend)

- Design Comment schema: taskId, authorId, body, createdAt, editedAt
- Build comment CRUD with author-only edit/delete permission
- Stub Attachment schema (placeholder URL — real S3 wiring is Week 3)

DEFINITION OF DONE — Comments persist, scoped correctly per task; only the author can edit/delete their own comment (verified).

DAY 11

Mon, Jul 13

Comments UI & Activity Logs

- Build CommentThread component
- Build a generic, append-only ActivityLog service triggered on key events
- Build ActivityFeed UI on the ProjectDetail page

DEFINITION OF DONE — Activity feed shows real chronological events across at least 5 distinct action types.

DAY 12

Tue, Jul 14

Filtering, Searching & Pagination

- Add query-param filtering to the task list endpoint (status, assignee, priority)
- Add cursor-based pagination (reuse the pattern already built for DevCollab search)
- Build FilterBar UI + load-more / infinite scroll

DEFINITION OF DONE — Filters and pagination work together without breaking cursor state; verified against 50+ seeded tasks.

DAY 13

Wed, Jul 15

Aggregation & Dashboard Metrics

- Write an aggregation pipeline for project stats (tasks by status, overdue count, activity count)
- Build a stats panel on Dashboard/ProjectDetail from the aggregated data

DEFINITION OF DONE — Stats panel numbers match a manual raw-query cross-check exactly.

DAY 14

Thu, Jul 16

■ WEEK 2 GATE — Integration, Testing & Buffer

- Full regression: task lifecycle create → assign → comment → status change → complete
- Fix bugs, refactor, update README
- git tag week2-complete

DEFINITION OF DONE — GATE: every Week 2 roadmap item is functional; task module usable end-to-end with zero errors.

WEEK 3 — COLLABORATION — Jul 17 to Jul 23, 2026

DAY 15

Fri, Jul 17

Socket.IO Server Setup

- Stand up the Socket.IO server with JWT auth on the handshake
- Implement per-project room-join logic
- Build presence tracking (Redis or in-memory map for now)

DEFINITION OF DONE — Authenticated socket joins the correct project room; presence list updates live on connect/disconnect.

DAY 16

Sat, Jul 18

Project Chat — Backend & Socket Events

- Design Message schema: projectId, senderId, body, createdAt, readBy[]
- Implement send-message, receive-message, typing-indicator events
- Persist every message to MongoDB on send

DEFINITION OF DONE — A message sent from one client appears in real time on another client in the same room, correctly persisted.

DAY 17

Sun, Jul 19

Chat UI

- Build ChatWindow, MessageBubble, TypingIndicator components
- Wire the Socket.IO client with reconnect handling
- Auto-scroll + read-receipt indicator

DEFINITION OF DONE — Chat is usable across two simultaneous browser sessions, live, with no duplicate messages.

DAY 18

Mon, Jul 20

Notifications System

- Design Notification schema + a notification-creation service triggered by key events
- Emit a socket event on notification creation
- Build NotificationBell + NotificationDropdown with mark-as-read

DEFINITION OF DONE — Triggering a tracked event (e.g. task assignment) produces a live notification with no page refresh.

DAY 19

Tue, Jul 21

AWS S3 File Uploads

- Set up an S3 bucket + least-privilege IAM policy + presigned URL endpoint
- Wire upload flow: request presigned URL → upload directly to S3 → save URL to DB
- Add file-type and size validation

DEFINITION OF DONE — A file uploaded from the UI lands in S3 and displays correctly via its saved URL.

DAY 20

Wed, Jul 22

Stripe Subscription & Free Plan Limit

- Integrate Stripe Checkout for subscription upgrade
- Enforce the 5-free-project limit server-side, not just in the UI
- Handle the checkout.session.completed webhook to update the Subscription collection

DEFINITION OF DONE — 6th project creation is blocked with an upgrade prompt; a completed Stripe test checkout unlocks unlimited projects via webhook.

DAY 21

Thu, Jul 23

■ WEEK 3 GATE — Integration, Testing & Buffer

- Full regression across chat, notifications, uploads and subscription flow together
- Manually load-test Socket.IO with several concurrent connections
- git tag week3-complete

DEFINITION OF DONE — GATE: real-time features are stable across multiple simultaneous sessions; no socket leaks on repeated connect/disconnect.

WEEK 4 — PRODUCTION — Jul 24 to Jul 30, 2026**DAY 22**

Fri, Jul 24

WebRTC Meetings — Signaling

- Implement Socket.IO signaling events (offer / answer / ICE candidates) per meeting room
- Build MeetingRoom backend: create, join, leave

DEFINITION OF DONE — Two clients complete a signaling handshake and reach a successful peer-connection state.

DAY 23

Sat, Jul 25

WebRTC Meetings — Media UI

- Build video/audio stream UI, mute + camera toggle, screen share
- Handle peer disconnect/reconnect edge cases

DEFINITION OF DONE — A two-participant video call works locally with mute, camera toggle and screen share all functioning.

DAY 24

Sun, Jul 26

Dockerization

- Write a multi-stage Dockerfile for the backend and a build+serve one for the frontend
- Write docker-compose.yml (backend, frontend, MongoDB, Redis)
- Verify the full stack boots via a single command

DEFINITION OF DONE — The entire app runs from ``docker compose up`` with zero manual setup steps.

DAY 25

Mon, Jul 27

Redis Integration

- Wire Redis as the Socket.IO adapter (multi-instance scaling readiness)
- Use Redis for session/presence caching

DEFINITION OF DONE — Socket.IO functions correctly with the Redis adapter enabled; setup documented if only single-instance is tested.

DAY 26

Tue, Jul 28

Nginx + AWS EC2 Deployment

- Configure Nginx as a reverse proxy (frontend/backend routing, SSL-ready)
- Provision an EC2 instance and deploy the Dockerized app
- Point a domain/subdomain if available

DEFINITION OF DONE — The application is publicly reachable via EC2 public IP or domain, served through Nginx.

DAY 27

Wed, Jul 29

CI/CD with GitHub Actions

- Write a workflow: lint → test → build → deploy on push to main
- Add a post-deploy health-check step

DEFINITION OF DONE — A push to main triggers an automated pipeline that deploys successfully with zero manual SSH steps.

DAY 28

Thu, Jul 30

Security & Performance Hardening

- Add helmet, tighten CORS, run an input-sanitization pass, confirm rate limiting on sensitive routes
- Add centralized error logging (e.g. Winston) and basic monitoring
- Performance pass: bundle size, route lazy-loading, confirm indexes are actually used

DEFINITION OF DONE — Full security checklist (Part III) ticked; zero unhandled promise rejections under normal use.

CLOSE-OUT — Jul 31 to Aug 01, 2026

DAY 29

Fri, Jul 31

■ WEEK 4 GATE — Full System QA

- Manually walk every flow in the spec: auth → project → task → chat → notification → meeting → upload → subscription
- Bug bash — log and fix everything found, prioritized by severity
- Update README, API reference and architecture notes

DEFINITION OF DONE — GATE: zero critical/high bugs open; a full fresh-session user journey completes without errors.

DAY 30

Sat, Aug 01

■ Final Polish, Demo & Retrospective

- Record a short demo walkthrough for your portfolio/LinkedIn
- Write a retrospective: what was built, what was learned, what's deferred to v1.1+
- git tag v1.0-complete and push the final commit
- Update portfolio/LinkedIn with the project and demo link

DEFINITION OF DONE — Demo recorded, retrospective written, repo tagged — DevCollab's 30-day build is formally closed.

PART III

Execution Appendix

Checklists to run at the two hardest gates.

PART III — APPENDIX
SECTION 12

Weekly Gate Summary

Day	Gate	Pass Condition
Day 7	Week 1 Gate	Auth, Profile, Project CRUD, Members, Invitations, RBAC, Dashboard all working end-to-end
Day 14	Week 2 Gate	Full task lifecycle, comments, activity logs, filtering/search/pagination working end-to-end
Day 21	Week 3 Gate	Chat, notifications, S3 uploads, Stripe subscription all stable under multi-session testing
Day 28-29	Week 4 Gate	WebRTC meetings, Docker, Redis, Nginx/EC2 deploy, CI/CD, security hardening — full QA pass
Day 30	Final Close-Out	Demo recorded, retrospective written, repo tagged v1.0-complete

PART III — APPENDIX
SECTION 13

Security & Hardening Checklist (Day 28)

- helmet enabled with sane defaults
- CORS locked to known frontend origin(s) only
- Rate limiting on auth, invitation, and payment-sensitive routes
- All user input validated server-side (express-validator / Zod), never trust client-side checks alone
- JWT secrets, DB URIs, Stripe keys, AWS keys all in environment variables — never committed
- Centralized error handler — no raw stack traces leaked to the client in production
- Winston (or equivalent) logging wired for server errors and key auth events
- MongoDB indexes confirmed in use for hot queries (projectId+status, projectId+userId, etc.)
- No unhandled promise rejections or floating async calls in server logs under normal use

PART III — APPENDIX
SECTION 14

Non-Negotiables

These apply every single day of the 30, without exception:

- A day is not “done” because time ran out — it's done when its Definition of Done is met.
- If a day's scope turns out to be too big, cut the polish, not the DoD.
- Every Gate is a hard stop. Do not carry unresolved Gate criteria into the next week.
- Document decisions as you go — the README updates on Days 7, 14, 21 and 29 are not optional.